Doramagic v3 产品定义手册（修订版）

> *日期: 2026-03-13*
> *状态: 修订版——从研究讨论过渡到开发准备*
> *演进: v3 初稿 → 10 轮审查讨论 → 三方研究 → 本修订版*
> *两个维度: Part I 用户视角（产品是什么）/ Part II 技术视角（怎么实现）*

Part I: 用户视角

> *这部分回答：用户看到什么、体验什么、得到什么。*
> *读者：产品经理、设计师、以及需要理解产品边界的开发者。*

---

1. 一句话定义

Doramagic 是运行在 OpenClaw 上的哆啦A梦——用户说出一个模糊的烦恼，Doramagic 从开源世界找到最好的作业，提取智慧，锻造出一个开袋即食的 AI 道具。

用户不需要知道"作业"是什么、从哪来、怎么提取。他只需要说出烦恼，拿到道具，用起来。

---

2. 产品哲学

2.1 产品设计之魂（不可修改）

> ***不教用户做事，给他工具。***

哆啦A梦从不教大雄怎么做事——掏出道具，交给大雄，大雄自己解决问题。Doramagic 的每一个产品决策都受这条哲学约束。

2.2 核心定位

Doramagic 是**抄作业的王者**。

- 不发明、不创造——在开源世界找到最好的现有方案，为用户量身适配
- LLM 的知识可以认为是无限的，用户的认知是有限的
- Doramagic 的价值 = 把 LLM 散落各处的知识**正确组装**成用户能用的道具
- 就像装配线上的熟练工——每个零件都不是他造的，但他知道怎么拼成产品

2.3 知识魔法，不是方法论魔法

> *Doramagic 注入能力，不强加流程。*

生成的道具不是固定步骤的执行器，而是注入了领域知识的 AI 专家。AI 自己判断什么时候走完整流程、什么时候直接干。

2.4 有立场的专家

大多数 AI 产品不敢做判断——列出三个选项说"取决于你的需求"。这是推卸责任。用户要的是成品，成品意味着有人替你做了判断。Doramagic 必须替用户做选择。

2.5 能力显性

> *Doramagic 的能力不能藏着。用户需要看到它在工作。暗默能力 = 不信任。*

- 环境感知要让用户看到（"我发现你已经安装了 xxx Skill，可以配合使用"）
- 提取过程要让用户看到进展（工厂透明）
- 道具的知识来源要可追溯（"这条规则来自 xxx 项目的代码"）

---

3. 用户旅程一：发明工具

> *用户通过 Doramagic 从零"发明"一个 AI 道具的完整过程。*

3.1 总览

```
用户："Dora，帮我做一个能追踪 A 股行情的东西"
      |
      ▼
Phase 1：需求挖掘（苏格拉底对话，3-5 轮）
  "你最想解决什么？"
      |
      ▼
Phase 2：发现作业（搜索开源项目 + 资源）
  "找到了 3 个好作业"  ← 用户可见
      |
      ▼
Phase 3：提取灵魂（Soul Extractor 工作）
  "正在提取智慧..."   ← 用户可见进度
      |
      ▼
Phase 4：锻造道具（知识编译 + 组装）
  "为你量身定制中..."  ← 用户可见进度
      |
      ▼
Phase 5：交付验证（测试 + 安装 + 引导）
  "道具准备好了！试试说'帮我看看今天A股行情'"
```

3.2 Phase 1: 需求挖掘

触发：`/dora 我想追踪A股行情` 或自然语言

Doramagic 做什么：
- 苏格拉底式对话，每轮 1–2 个问题
- 问核心痛点、使用场景、成功标准、偏好约束
- 最终确认："我理解你需要 xxx，对吗？"

用户感知：
- 像跟一个懂行的朋友聊天，不像填表
- Doramagic 会主动说"我看到你已经装了 xxx"（环境感知显性化）

硬门控：需求画像必须包含 `core_problem`、`trigger`、`expected_output`、`user_environment`，否则继续对话。

3.3 Phase 2: 发现作业

Doramagic 做什么：
– 四梯队搜索：MCP Server + GitHub + Skill 市场 → API 文档 + Stack Overflow → 技术博客 → 社区讨论
– 产出源泉地图，标注每个来源的价值

用户感知：
– 看到 Doramagic 在找什么、找到了什么
– 可以确认或补充："我之前用过 xxx，你也看看"

> *"我找到了 3 个相关项目：akshare（A 股数据接口）、mootdx（通达信行情）、quantaxis（量化框架）。你想让我从哪些中提取智慧？"*

3.4 Phase 3: 提取灵魂

Doramagic 做什么：六层知识提取（细节见 Part II）

用户感知：
– 工厂透明——看到生产流水线在运转
– "正在分析项目结构..."
– "正在提取设计哲学..."
– "正在交叉验证社区经验..."
– "知识提取完成，开始锻造道具"

3.5 Phase 4: 锻造道具

Doramagic 做什么：Knowledge Compiler 编译 + 叙事合成（细节见 Part II）

用户感知：
– "正在为你定制道具..."
– 等待时间估计：3–10 分钟（取决于项目规模和知识源数量）

3.6 Phase 5: 交付验证

三个步骤：
1. **冒烟测试**：Doramagic 自动验证道具格式正确、资源可达、触发词有效
2. **安装**：将道具安装到用户的 OpenClaw workspace
3. **极简引导**：告诉用户三件事——
– 能做什么（一句话能力概述）
– 怎么触发（触发词）
– 能力边界（不能做什么）

> *"你的 A 股行情追踪助手已经准备好了！*
> *它能帮你查实时行情、分析个股、追踪自选股。*
> *直接说'帮我看看今天 A 股行情'就能用。*
> *注意：它用的是 akshare 数据源，盘后数据有 15 分钟延迟。*
> *要不要现在试一下？"*

极简引导不违反"不教做事"——它告诉用户道具的能力边界，不是教用户怎么用。就像哆啦A梦说"这是任意门，想去哪就开门"，不是在教大雄走路。

---

4. 用户旅程二：使用工具

4.1 道具自解释

道具不需要说明书。
- 触发词是用户日常会说的话（"帮我看看行情"不是"/run stock-tracker"）
- 首次触发时道具用一句话介绍自己
- 基础功能直接可用，高级功能在需要时自然浮现

4.2 道具 vs 通用 AI

场景	通用 AI	Doramagic 道具
"帮我分析茅台"	给一段网上都能找到的分析	用 akshare 拿实时数据，知道 PE/PB 分析的正确做法，知道数据陷阱
遇到边界情况	"这取决于你的需求"	有明确立场，基于验证过的决策规则做判断
犯错概率	高（依赖通用知识）	低（有验证过的领域知识锚定）

4.3 回炉改造

用户主动要求修改道具功能：

> *"Dora，帮我改一下行情追踪器，加上港股支持"*

Doramagic 读取原道具 + 使用记忆 → 增量提取港股知识 → 重新编译 → 道具 v1.0 → v1.1。保留所有使用中积累的个性化数据。

4.4 道具自动更新（影子锻造）

Doramagic 自身升级后（引擎更强、积木更多），旧道具应该受益：

1. 每个道具内嵌"出生证明"（引擎版本、积木版本、锻造时间）
2. 用户下次触发 Doramagic 时，自动扫描已安装道具
3. 发现可更新的道具 → 后台影子锻造新版本
4. 推送用户确认："你的行情追踪器有新版本可用，要更新吗？"
5. 用户确认后替换，保留 MEMORY.md 个性化数据
6. 支持一键回滚

用户不需要知道 Doramagic 的版本号或技术升级细节。

4.5 多道具协作

用户可能有多个 Doramagic 道具。它们可以：
– 互相调用（行情追踪器调用选股器）
– 在对话中自然组合使用
– 通过 OpenClaw 的 Skill 间调用能力和共享 workspace 实现

---

5. 品牌与交互

5.1 品牌映射

哆啦A梦	Doramagic
大雄带着烦恼来	用户带着模糊需求触发 /dora
哆啦A梦理解真实需求	苏格拉底对话
从口袋里掏出道具	从开源世界提取智慧，锻造道具
道具交给大雄自己用	安装到 OpenClaw，用户直接用
道具被滥用时警告	透明展示能力边界
大雄遇到新问题再来	回炉改造 + 影子锻造

5.2 人设

亲切、直接、偶尔吐槽。不教育用户，不炫技术。像一个靠谱的朋友。

5.3 触发方式

- `/dora` + 需求描述
- "Dora，帮我 XXX"
- 自然语言触发

---

6. 价值主张

6.1 价值公式

> *Doramagic 的价值 = LLM 已有知识 × 操作化组装能力*

LLM 知道很多，但散落各处。用户不知道怎么找、怎么组合、怎么用。Doramagic 把这些散落的知识正确组装成一个可用的道具。

6.2 用户价值

用户类型	没有 Doramagic	有 Doramagic
老王（散户）	问 AI"帮我炒股"得到泛泛而谈	得到一个真正懂 A 股数据的专家道具（30→60 分）
专业用户	自己配置各种 API 和工具	得到一个注入了最佳实践的精准工具（80→85 分）

6.3 护城河

核心护城河：WHY + UNSAID 提取能力——行业空白。

没有任何现有工具能自动提取设计哲学和隐性知识。这是 Doramagic 的 Soul Extractor 独占的能力。

飞轮效应：积木仓库扩展 → 提取精度提升 → 道具质量提升 → 用户满意 → 更多使用反馈 → 积木仓库继续扩展。

Part II: 技术视角

> *这部分回答：怎么实现、组件边界、数据流、接口契约。*
> *读者：开发者——需要知道每个组件做什么、输入输出是什么、跟谁对接。*

7. 架构全景

7.1 两层分离

```
┌──────────────────────────────────────────────────┐
│                提取核心（平台无关）                  │
│                                                  │
│  git clone → 确定性提取 → LLM 提取 →              │
│  Knowledge Compiler → 知识卡片                    │
│                                                  │
│  这一层不依赖任何平台，可以在 CLI、OpenClaw、       │
│  或任何其他环境中运行                              │
├──────────────────────────────────────────────────┤
│              平台适配层（可替换）                   │
│                                                  │
│  OpenClaw 适配：                                  │
│  ├── SKILL.md 格式输出                            │
│  ├── sessions_spawn 子代理调度                    │
│  ├── MEMORY.md 持久化                             │
│  └── 渠道分发（WhatsApp/Telegram/WebChat）        │
│                                                  │
│  CLI 适配（fallback）：                           │
│  ├── CLAUDE.md 格式输出                           │
│  └── 本地文件系统                                 │
└──────────────────────────────────────────────────┘
```

设计原则：提取核心平台无关，OpenClaw First 是阶段性市场最优策略，不是架构约束。底层要解决的问题（提取、推理、编译、验证）在任何平台上都一样。

7.2 组件全景

```
确定性编排层（Shell / exec 工具）
|
├── Phase 路由（需求→发现→提取→锻造→交付）
├── 质量门控（每阶段硬门控）
├── 子代理调度（sessions_spawn / 本地进程）
|
├── Stage 0: 确定性提取 ──────────────── 无模型
|   ├── repo_facts.json 提取器
|   ├── 框架检测 → 积木加载器
|   └── 热力图生成器（概念，待验证）
|
├── Stage 1: WHY 提取 ──────────────────── Opus 子代理
|   输入: 积木 + 代码片段 + 热力图
|   输出: WHY 卡片 + 证据链
|
├── Stage 2-3: 概念/规则提取 ──────────── Sonnet 子代理
|   输入: 积木锚点 + 代码片段
|   输出: 概念卡 + 规则卡 + 证据链
|
├── Stage 3.5: 验证关卡 ──────────────── 确定性脚本
|   ├── fact-checking gate
|   ├── 证据链裁决（规则引擎）
|   └── 社区证据交叉验证
|
├── Knowledge Compiler ────────────────── 确定性编译
|   输入: 知识卡片 + 证据裁决
|   输出: AI 可消费的编译产物
|
└── Stage 4: 叙事合成 ──────────────────── Opus 子代理
    输入: 编译产物 + Skill 模板
    输出: 完整 Skill 包
```

7.3 数据流总览

```
GitHub URL
    |
    ▼
[Stage 0] ──→ repo_facts.json + framework_id + heatmap(概念)
    |
    ▼
[积木加载器] ──→ 匹配的积木集合（L1 + L2）
    |
    ├──→ [Stage 1: WHY] ──→ WHY 卡片[]
    ├──→ [Stage 2: 概念] ──→ 概念卡片[]
    └──→ [Stage 3: 规则] ──→ 规则卡片[]
              |
              ▼
       [Stage 3.5: 验证] ──→ 带证据裁决的卡片[]
              |
              ▼
       [Knowledge Compiler] ──→ 编译产物（按知识类型格式化）
              |
              ▼
       [Stage 4: 合成] ──→ SKILL.md（完整道具包）
              |
              ▼
       [冒烟测试 + 安装]
```

---

8. 平台策略

8.1 OpenClaw First

OpenClaw 是当前阶段的最优平台：
- 提供完整运行时（模型调用、工具编排、记忆、用户交互、50+ 渠道）
- Skill 生态 5400+，道具可调用已有 Skill
- 子代理支持（sessions_spawn，最多 5 并发）
- 市场窗口期——OpenClaw 崛起快，兵工厂不足，机会巨大

8.2 OpenClaw 硬性约束与适配

约束	适配策略
SOUL.md 加载不可靠	关键知识写入 SKILL.md content 区
子代理不继承 SOUL.md	指令通过 AGENTS.md 显式传入
默认模型 MiniMax（能力有限）	关键阶段显式指定 Opus/Sonnet
异步执行，无法实时监控	确定性检查点 + 质量门控
Skill 文件结构固定	严格遵循 SKILL.md + frontmatter + content

8.3 降级策略

子代理调度失败 → 自动降级为单代理串行模式（v1.0 管线作为 fallback）。确保在 OpenClaw 子代理不可用时仍能产出结果，只是质量和速度下降。

---

9. Stage 0: 确定性提取

> *无模型参与。纯脚本/AST 分析。输出是后续所有 LLM 阶段的基础事实。*

9.1 repo_facts.json

输入：git clone 后的项目目录

输出：确定性事实白名单

```
{
  "project_name": "akshare",
  "language": "Python",
  "framework": "none",
  "dependencies": ["requests", "pandas", "lxml"],
  "file_structure": { "src/": [...], "tests/": [...] },
  "commands": ["pip install akshare"],
  "api_signatures": ["stock_zh_a_spot_em()", "stock_hk_spot()"],
  "config_files": ["setup.cfg", "pyproject.toml"],
  "test_count": 142,
  "loc": 28500,
  "star_count": 8200,
  "last_commit": "2026-03-10"
}
```

关键规则：事实层零自由度——AI 不许编。所有事实必须可从代码确定性验证。后续 LLM 提取的任何事实声明必须在此白名单中存在，否则标记为"未验证"。

9.2 框架检测 → 积木加载

输入：repo_facts.json 中的 framework + dependencies

输出：匹配的积木 ID 列表

```
检测到 Django + Celery + Redis
  → 加载 django-core (L1) + django-orm (L2) + celery-core (L1)
  → 积木组合图谱：django + celery = "推荐组合"，加载组合积木
```

9.3 热力图（概念阶段，待实验验证）

目标：产出一张"抄作业优先级地图"，告诉后续子代理哪里值得深挖。

当前设计方向（三方头脑风暴共识，未验证）：
- 四种热度维度：共识热 / 杠杆热 / 差异热 / 踩坑热
- 输出不是分数，是可行动标签：`consensus_anchor` / `novel_pattern` / `risk_sink` / `protocol_hub` / `boilerplate`
- 偏离检测 = 黄金区域：积木定义标准做法，热力图标出偏离
- 动态演化：Stage 0 初始 → Stage 1 修正 → 用户反馈修正

验证要求：至少 3 个项目验证——2 个相似 + 1 个不相关。

当前实现：暂不实现完整热力图。先用简单的引用计数 + 文件变更频率作为 MVP 信号，在实验中逐步验证哪些维度有用。

---

10. Stage 1–3: LLM 提取

10.1 六层知识模型

层	含义	提取方式	模型自由度
领域知识	该知道什么	LLM + 积木锚定	AI 可以解读但不能瞎编
操作知识	怎么做	确定性 + LLM 补充	事实部分 AI 不许编
经验知识	什么有坑	社区证据交叉验证	AI 可以解读但不能瞎编
决策知识	何时用什么	LLM + 社区证据	AI 可以解读但不能瞎编
设计哲学	为什么这样	LLM 第一性原理推理	AI 可以解读但不能瞎编
隐性知识	没人说的事	社区信号 + LLM	AI 可以解读但不能瞎编

后两层（WHY + UNSAID）= Doramagic 护城河，行业空白。

10.2 Stage 1: WHY 提取

子代理：model: Opus（需要深度推理）

输入：
- 积木（≤2500 tokens）
- 代码片段（≤20K tokens，由热力图/引用计数指导选择）
- repo_facts.json

输出：WHY 卡片数组

```
- claim_id: W001
  layer: design_philosophy
  text: "该项目优先使用乐观重试而非熔断，因为其业务场景容忍短暂延迟但不容忍数据丢失"
  reasoning_chain: "1. retry_policy.go 实现了指数退避 2. 无熔断器代码 3. 配置中 max_retries=5 
  evidence_profile:
    - type: CODE
      source: src/retry/policy.go#L42-L78
      reproducible: true
    - type: DOC
      source: docs/ops.md#L88-L95
      reproducible: true
```

约束：
– 每条 WHY 必须附推理链（从代码出发推理）
– 每条 WHY 必须附证据（至少一条 CODE 或 DOC）
– 特别关注：项目偏离框架标准的地方——这些偏离往往是最重要的设计决策

10.3 Stage 2–3: 概念/规则提取

子代理：model: Sonnet + 积木锚点

输入：
– 积木锚点（≤1000 tokens，只含积木的关键概念列表）
– 代码片段（≤15K tokens）

输出：概念卡 + 规则卡

```yaml
# 概念卡
- claim_id: C001
  layer: domain_concept
  text: "akshare 将数据源抽象为 '接口函数', 每个函数对应一个数据端点"
  evidence_profile:
    - type: CODE
      source: akshare/stock/stock_zh_a_spot_em.py
      reproducible: true

# 规则卡
- claim_id: R001
  layer: decision_rule
  text: "获取实时行情时应使用 stock_zh_a_spot_em(), 不要使用 stock_zh_a_spot() (已弃用) "
  causal_annotation: "旧接口 stock_zh_a_spot() 依赖的数据源已停止维护"
  evidence_profile:
    - type: CODE
      source: akshare/__init__.py (deprecation warning)
      reproducible: true
    - type: COMMUNITY
      source: github_issue_1234
      reproducible: true
```

关键改进（来自三方研究）：规则卡必须带因果注释（`causal_annotation`）。规则从叙事中抽离后丢失因果上下文是 v0.8→v0.9 降分的根因。

---

11. Stage 3.5: 验证关卡

> *确定性脚本执行。LLM 不参与裁决。*

11.1 事实验证

所有卡片中引用的事实（文件名、API 名、命令）必须在 repo_facts.json 白名单中存在。不存在的 → 标记为"未验证"。

11.2 证据链裁决系统

替代原 A/B/C/D 主观评级（三方研究共识）。

核心思想：LLM 是检察官（提出主张 + 附上证据），不是法官。法官是确定性规则引擎。

证据类型：
- `[CODE]` — 代码中直接可观察
- `[DOC]` — 项目文档明确说明
- `[COMMUNITY]` — 社区讨论中提到
- `[INFERENCE]` — LLM 推断，无外部证据

裁决规则（版本化，可迭代）：

```
policy_v1:
  R1: ≥1 CODE 或 ≥1 DOC 支持，无高优先级冲突 → SUPPORTED
  R2: 仅 COMMUNITY 支持，独立来源 ≥2 → WEAK
  R3: 仅 INFERENCE → WEAK + QUARANTINE
  R4: CODE 与 DOC 直接矛盾 → CONTESTED
  R5: 可执行检查反证（如测试失败）→ REJECTED
```

来源优先级：`runtime CODE > static CODE > DOC > COMMUNITY > INFERENCE`

裁决输出：
- `verdict`: SUPPORTED / CONTESTED / WEAK / REJECTED
- `policy_action`: ALLOW_CORE / ALLOW_STORY / QUARANTINE

冲突处理：
1. 锁定 repo_snapshot_sha，所有证据绑定同一版本
2. 按来源优先级 + 时间戳排序
3. 仍有冲突 → CONTESTED，生成最小验证脚本
4. 无法自动解决 → 保留冲突态，不进入 ALLOW_CORE

11.3 高 Star vs 低 Star 项目适配

- **高 Star**：强制仓库内证据闭环，LLM 预训练记忆只做候选不入库，提高反证检查权重
- **低 Star**：允许更多 INFERENCE 进 ALLOW_STORY（不进 ALLOW_CORE），增加代码结构证据挖掘深度

原则：框架统一，阈值自适应。不因热度改变真值定义，只改变可接受风险边界。

---

12. Knowledge Compiler

> *确定性编译。源格式（结构化卡片）→ 消费格式（AI 可消费的混合体）。不增删内容，只做格式转换。*

12.1 编译规则

按知识类型路由格式：

知识类型	编译格式	理由
事实型（API、命令、配置）	结构化列表	精确引用，无歧义
规则型（决策规则、最佳实践）	规则语句 + 因果注释	保留因果上下文（解决 v0.8→v0.9 降分）
哲学型（设计意图、心智模型）	纯叙事	LLM 最擅长理解叙事体
踩坑型（社区经验、反模式）	案例故事	经验知识用故事最有效

12.2 按证据裁决过滤

裁决	处理
SUPPORTED	纳入，可进入能力规则和关键决策
WEAK	降为注释或进入"踩坑经验"区，标注"社区经验，未官方确认"
CONTESTED	保留冲突双方，标注"存在争议"
QUARANTINE	不纳入道具，存档待验证
REJECTED	丢弃

12.3 输出格式

按 Lost in the Middle U 型注意力分布排列：

```
开头（高注意力区）：哲学 + 关键规则
中间（注意力衰减区）：踩坑故事（叙事体缓解衰减）
尾部（高注意力区）：能力清单 + 速查表
```

总 token 预算：≤2000 tokens（叙事 ~40% + 结构化 ~50% + 元数据 ~10%）

12.4 证据脚注

编译产物中的关键知识附最小证据脚注：

```
- 默认采用指数退避重试（来源：CODE: `src/retry.go:42`；DOC: `docs/ops.md:88`）
- 社区反馈该策略在高并发下可能放大尾延迟（来源：COMMUNITY: issue #348，未官方确认）
```

---

13. Stage 4: 叙事合成

子代理：model: Opus

输入：
– Knowledge Compiler 编译产物（≤5K tokens）
– SKILL.md 模板（≤500 tokens）
– 无项目代码（已被编译产物替代）

输出：完整 SKILL.md

```
# 生成的 Skill 文件结构
my-stock-tracker/
├── SKILL.md              # 主文件
├── references/           # 参考文档（可选）
└── _meta.json            # ClawHub 元数据
```

SKILL.md 内容结构：

```
---
name: my-stock-tracker
description: >
  A 股行情追踪与分析助手。
  Triggers on: "行情", "股票", "A股", "涨跌", "stock"
user-invocable: true
metadata:
  openclaw:
    category: finance
  doramagic_origin:           # 出生证明（影子锻造用）
    tool_id: "stock-tracker-2026-abc"
    forge_timestamp: "2026-03-13T10:00:00Z"
    engine_version: "v3.0.5"
    compiler_version: "v1.2.0"
    bricks_used:
      - {id: "akshare-core", version: "v1.0.0"}
---

# A 股行情追踪助手

## 你是谁
一句话定位 + 心智模型类比

## 设计哲学
纯叙事，2-3 个因果链

## 关键规则
≤5 条，每条 = 规则 + 因果注释 + 证据脚注

## 踩坑经验
2-3 个案例故事

## 能力清单
结构化列表

## 速查表
≤8 行 Markdown 表格
```

14. 灵魂积木系统

14.1 积木规格

层级	范围	Token 预算	示例
L1 框架级	框架核心哲学	200–400 tokens	django-core
L2 模式级	特定模式设计意图	400–800 tokens	django-orm

总注入预算：≤2500 tokens

14.2 积木的双重作用

1. **锚定**：框架公共知识已由积木覆盖，提取引擎跳过不重复提取
2. **过滤**：偏离框架核心设计太远的提取结果大概率是噪音

第三重作用（偏离检测基准线）：积木定义"标准做法"，跟项目实际代码的 diff = 项目独特的设计决策 = 提取的黄金区域。

14.3 积木仓库设计

不是扁平列表，是有结构的图（三方共识）：

边类型	含义	示例
`contains`	包含关系	django-core → django-orm
`composes_with`	推荐组合	django + celery
`substitutes`	替代关系	django-orm ↔ sqlalchemy
`conflicts_with`	互斥	—

优先级排序（冷启动阶段）：
- A 类（高频）：django, react, express → 优先制作
- B 类（中频）：celery, redis, sqlalchemy → 按需制作
- C 类（长尾）：不做积木，靠 LLM 通用知识

保鲜期 TTL：积木元数据包含知识衰减速度。React hooks 最佳实践 ~6 个月更新；Unix 哲学 ~永不过期。

反积木（Anti-pattern Bricks）：记录"千万别这么做"。用于负向锚定——在项目中检测到反模式时，作为踩坑经验重点提取。

14.4 积木 schema

```
brick_id: django-orm
level: L2
framework: django
version: "1.0.0"
ttl: "1y"                    # 保鲜期
token_count: 620
dependencies: [django-core]   # 图结构
substitutes: [sqlalchemy-core]
composes_with: [celery-core]

# 内容
standard_patterns:            # 标准做法（用于偏离检测）
  - "使用 ModelForm 处理表单"
  - "使用 select_related/prefetch_related 优化查询"
  - "使用 Django migration 管理数据库变更"

anti_patterns:                # 反模式
  - "在循环中调用 .save()"
  - "N+1 查询"
  - "用 eval() 做动态查询"

content: |
  Django ORM 的核心哲学是"显式优于隐式"...
  [叙事体，包含设计意图和因果链]

evidence_sources:             # 积木来源可追溯
  - "Django 官方文档 v5.0"
  - "Two Scoops of Django"
  - "验证项目: djangoproject.com, wagtail, saleor"
```

---

15. 子代理架构

15.1 模型路由

阶段	模型	理由
编排/调度	无（确定性）	关键步骤不依赖模型
需求对话	Sonnet	对话理解足够
源泉发现	Sonnet	搜索筛选不需要深度推理
WHY/UNSAID 提取	Opus	需要深度推理
概念/规则提取	Sonnet + 积木	有积木辅助足够
叙事合成	Opus	高质量叙事需要深度语言能力
冒烟测试	Sonnet	验证任务不需要深度推理

15.2 上下文预算

每个子代理独立上下文，不互相挤压：

阶段	内容	预算	不需要
Stage 1	积木(≤2500) + 代码(≤20K)	~23K	SKILL.md
Stage 2–3	积木锚点(≤1000) + 代码(≤15K)	~16K	SKILL.md
Stage 4	编译产物(≤5K) + 模板(≤500)	~6K	项目代码

这是调度问题，不是预算问题——没有任何一个阶段需要同时塞入全部知识。

15.3 降级策略

```
正常模式：确定性编排 → 多子代理并行
    |
    如果子代理调度失败
    |
    ▼
降级模式：单代理串行（v1.0 管线）
    |
    质量可能下降，但保证有产出
```

15.4 OpenClaw 子代理实现要点

- 子代理不继承 SOUL.md → 关键指令必须写 AGENTS.md 或显式传入
- 最多 5 并发 → Stage 1/2/3 可并行，Stage 4 串行
- 异步执行 → 每个阶段完成后写中间产物到 workspace，下一阶段从文件读取
- 中间产物格式：YAML 卡片文件（知识卡片、证据链、裁决结果）

---

16. 影子锻造（道具更新机制）

16.1 出生证明

每个道具内嵌元数据：

```yaml
doramagic_origin:
  tool_id: "stock-tracker-2026-abc"
  forge_timestamp: "2026-03-13T10:00:00Z"
  engine_version: "v3.0.5"
  compiler_version: "v1.2.0"
  bricks_used:
    - {id: "akshare-core", version: "v1.0.0"}
  source_repos:
    - {url: "github.com/akfamily/akshare", commit: "abc123"}
  user_intent_snapshot: "A股实时行情追踪，核心需求是自选股监控"
  customization_hash: "h123456"  # MEMORY.md 的 hash
```

16.2 更新流程

```
用户触发 Doramagic（任何操作）
    |
    ▼
扫描 workspace 中所有带 doramagic_origin 的 Skill
    |
    ▼
比对 engine_version / compiler_version / bricks_version
    |
    如果发现版本差异
    ▼
后台影子锻造新版本（用户无感知）
    |
    ▼
推送确认："你的 xxx 有新版本，要更新吗？"
    |
    ├── 用户同意 → 替换，保留 MEMORY.md
    └── 用户拒绝 → 不动，下次不再提醒（直到下一次版本差异）
```

16.3 回滚

更新前自动备份旧版本。用户可一键回滚："Dora，回到上一个版本"。

---

17. 质量保证体系

17.1 四层门控

门控	阶段	检查内容	失败处理
需求完整性	Phase 1 后	need_profile 包含全部必填字段	继续对话
源泉充分性	Phase 2 后	≥1 主力源泉 + ≥1 资源源泉	回退补充搜索
提取质量	Phase 3.5	≥3 张 SUPPORTED 卡片 + 事实验证通过	换源泉或重试
道具可用性	Phase 5	Skill 格式正确 + 触发词有效	回退 Phase 4 修复

17.2 提取噪音控制

噪音类型	危害	解法
幻觉（错误信号）	致命	三层防护：第一性原理约束 + 社区交叉验证 + 项目规模调节
不相关事实	高	积木减少搜索面 + 热力图指导优先级
冗余信息	中	Knowledge Compiler 去重压缩
过度解释	中	token 预算硬约束

17.3 模块化验证原则

每个组件应能独立验证（验证策略是架构质量的试金石）：

组件	独立验证方法
repo_facts 提取器	对照人工标注的 ground truth
积木质量	专家审核 + A/B 对比（有积木 vs 无积木）
WHY 提取	人工评估推理链质量 + 幻觉率
证据裁决引擎	删证据单调性测试 + 注入伪证据鲁棒性测试
Knowledge Compiler	有/无编译 A/B 对比任务成功率
最终道具	端到端任务成功率 + 用户纠错次数

---

18. 接口契约

> *组件间的输入输出规范。开发时每个组件按契约实现，独立可测。*

18.1 Stage 0 → Stage 1/2/3

```
# Stage 0 输出
repo_context:
  repo_facts: RepoFacts          # 确定性事实
  framework_id: string           # 检测到的框架 ID
  matched_bricks: BrickRef[]     # 匹配的积木引用
  heatmap: HeatmapEntry[]        # 热力图（MVP: 引用计数+变更频率）
  code_snippets:                 # 按热力图选择的代码片段
    - path: string
      content: string
      heat_label: string         # consensus_anchor / novel_pattern / ...
```

18.2 Stage 1/2/3 → Stage 3.5

```
# 知识卡片（统一格式）
knowledge_card:
  claim_id: string
  layer: enum[domain|operation|experience|decision|philosophy|implicit]
  text: string
  reasoning_chain: string        # 仅 WHY 卡片
  causal_annotation: string      # 仅规则卡片
  evidence_profile:
    - evidence_id: string
      type: enum[CODE|DOC|COMMUNITY|INFERENCE]
      source: string             # 可定位的引用
      reproducible: boolean
```

18.3 Stage 3.5 → Knowledge Compiler

```
# 带裁决的卡片
judged_card:
  card: KnowledgeCard
  verdict: enum[SUPPORTED|CONTESTED|WEAK|REJECTED]
  policy_action: enum[ALLOW_CORE|ALLOW_STORY|QUARANTINE]
  policy_version: string
  conflicts: ConflictRecord[]    # 如果 CONTESTED
```

18.4 Knowledge Compiler → Stage 4

```
# 编译产物
compiled_knowledge:
  philosophy_section: string     # 纯叙事
  rules_section: string          # 规则 + 因果注释
  traps_section: string          # 案例故事
  capabilities_section: string   # 结构化列表
  quickref_section: string       # 速查表
  total_tokens: int              # ≤2000
  evidence_footnotes: string[]   # 证据脚注
```

18.5 Stage 4 → 最终产出

```
# Skill 包
skill_package:
  skill_md: string               # 完整 SKILL.md 内容
  references: File[]             # 参考文档
  meta_json: object              # ClawHub 元数据
  doramagic_origin: Origin       # 出生证明
```

---

19. 开发路线图

19.1 MVP（最小可验证产品）

```
Step 1: 积木冷启动
  → 制作 django-core + django-orm 积木
  → 验证积木对提取质量的提升

Step 2: Soul Extractor v2
  → 实现确定性编排 + 子代理调度
  → 实现证据链裁决系统
  → 在已有 benchmark 上验证

Step 3: 端到端验证
  → 从 "帮我做一个 Django 管理后台" 到生成可用 Skill
  → 在 OpenClaw 中实际运行

Step 4: 跨领域验证
  → 从 Django 扩展到其他领域（金融/健身）
  → 验证积木 + 引擎的通用性
```

19.2 需要在实验中验证的假设

假设	验证方法	影响
积木能提升提取质量	有积木 vs 无积木 A/B	如果不能，积木系统需要重新设计
证据链裁决比 A/B/C/D 更准	对比实验	如果不能，退回 A/B/C/D
子代理架构比单代理好	对比实验	如果不能，简化为单代理
Knowledge Compiler 提升消费质量	有编译 vs 无编译 A/B	如果不能，简化格式
热力图维度有区分度	3 个项目验证	决定保留哪些维度

附录 A: 硬性约束清单

1. **第一性原理**：所有技术决策从根本事实推导

2. **产品设计之魂**：不教用户做事，给他工具
3. **代码说事实，AI 说故事**：事实层 AI 不许编，解读层 AI 可以解读但不能瞎编
4. **OpenClaw 适配**：Doramagic 适应平台，不可反过来
5. **Knowledge Compiler 不增删内容**：只做格式转换
6. **能力显性**：Doramagic 的能力让用户看得见

附录 B: 三方研究索引

研究	路径	核心结论
灵魂积木粒度	research/soul-lego-bricks/reports/synthesis-report.md	L1(200-400) + L2(400-800)，总≤2500
AI 知识消费格式	research/ai-knowledge-consumption/reports/synthesis-report.md	结构化抽取，叙事化编译，分块式注入
置信度体系	research/product-definition-v3/confidence-system-*.md	证据链 + 确定性裁决（三方共识）
道具更新机制	research/product-definition-v3/tool-update-mechanism-gemini.md	影子锻造 + 出生证明
热力图	research/product-definition-v3/heatmap-brainstorm-*.md	四种热度，待实验验证

附录 C: 5 大技术难点索引

难点	解决方案	详细记录
1. 幻觉与证据绑定	三层防护 + 证据链裁决	memory/doramagic-5-challenges.md
2. 确定性与生成式边界	Knowledge Compiler + 按类型路由	+ synthesis-report.md
3. 跨模型/Agent 控制	确定性编排 + 子代理路由	+ OpenClaw 多代理研究
4. 知识密度与可用性	积木过滤 + 子代理调度	+ AI 知识消费研究
5. 多源融合与端到端	证据链 + 标签脉络 + 集成	—

附录 D: 审查结论索引

问题	结论
1. 合理 vs 已验证	模块化验证，验证策略是架构质量试金石
2. 信息漏斗	真现象非核心，核心是拉高作业下限与上限
3. 降级策略	子代理失败 → 单代理串行 fallback
4. OpenClaw First	阶段性市场最优，核心提取引擎平台无关
5. 积木确认偏误	加负向约束"找偏离"，具体平衡点留实验
6. 置信度体系	证据链 + 确定性裁决（三方共识）
7. Token 预算	可配置，留给实验
8. 合规法律	初期不管
9. 多道具冲突	后续解决
10. /trace 调试	初期不需要，影子锻造解决更新